

Extending an existing application using a Plugin

We will now add a new plugin your application that we will run by clicking a button available in the top toolbar of our newly created application (this chapter starts with the final state of chapter [Creating an application configuration file](#)).

At first, our plugin will only open a new tab in your browser, but in the next chapter we will iterate over it.

We will start by creating a new file that will contain the button code:

1. In VS Code, within your Modules folder, go to the custom/public/js folder.
2. Create a new folder called NewPlugin

The folder is just for organization purposes, the file could be created in the custom/public/js folder. But since we will create more files it's best to organize them within a folder

3. Within the newly create folder, create a new file and call it newPlugin.js.
4. Add the code below to the file and save it.

[Copy](#)

```
import { Plugin, PluginButton } from 'myWorld-client'; //We'll use the PluginButton class to create our
//Your plugins will always extend the myw.Plugin class
class NewPlugin extends Plugin {
  static {
    this.prototype.buttons = {
      //First you define the buttons that will be shown in the toolbar. Each button has an ID and a
      dialog: class extends PluginButton {
        static {
          this.prototype.titleMsg = 'newplugin_msg'; //This is the tooltip for the button. We'
          this.prototype.imgSrc = newPluginImg; //This is the button image
        }
        action() {
          //and this is the action to be executed by the plugin. For now only opening a new tab
          window.open('https://www.iqgeo.com');
        }
      }
    };
  }
  constructor(owner, options) {
    super(owner, options);
  }
}
export default NewPlugin;
```

Now we will set the image that will be shown in the toolbar button. You can use any image you want.

5. Copy the icon image file file to the images folder.

Within VS Code the folder will be under modules/public/images.

In Explorer/Finder, the folder will be under <Project Folder>/custom/public/images.

6. Add the import line to the newPlugin.js file, your code should look like this:

[Copy](#)

```
import { Plugin, PluginButton } from 'myWorld-client';
import printImg from '../images/toolbar/print.svg'; //Newly added image import. Make sure to use the sam
class NewPlugin extends Plugin {
  static {
    this.prototype.buttons = {
      dialog: class extends PluginButton {
        static {
          this.prototype.titleMsg = 'newplugin_msg';
          this.prototype.imgSrc = newPluginImg;
        }
        action() {
          window.open('https://www.iqgeo.com');
        }
      }
    };
  }
  constructor(owner, options) {
    super(owner, options);
  }
}
```

```

    }
    constructor(owner, options) {
        super(owner, options);
    }
}
export default NewPlugin;

```

7. To add your new toolbar button to your application file, open the application configuration file created previously and add or update the following code sections:

```

import NewPlugin from './plugins/newPlugin';

// Add / configure plugins
var plugins = myw.applicationDefinition.plugins;
plugins['new_plugin'] = NewPlugin;

// Config desktop layout
var desktopLayoutDef = myw.applicationDefinition.layouts.desktop;
var desktopToolbarButtons = desktopLayoutDef.controls.toolbar[1].buttons;

desktopToolbarButtons.push('new_plugin.dialog');

```

[Copy](#)

You should now see your new toolbar button in your application, and clicking on it will open a new tab in your browser.



Figure: Additional button added to the interface, when clicked the button calls the action function from the Plugin

Copyright © 2026, IQGeo UK Ltd. IQGeo is a registered ® trademark.